Exercise 1: Evaluate The Conditions

```
int x = 8, y = 3, z = 8;
bool ready = false;
```

Complete the table without running a program. Write both the Boolean result and its numeric value (0 or 1)

	Expression	True/False	0/1
a	x > y	True	1
b	x != z	False	0
c	(y >= 1) && (y <= 5)	True	1
d	(x < y) || (z == 8)	True	1
e	!ready	True	1
f	(x < y) || (z == 8) && (y > 5)	False	0

g) Explain which logical operator is evaluated first in (f).

x<y = false

z==8 = true

y>5 = false

False || (true && false) = false

Exercise 2 : Trace Independent if statements

Assume the following code is inside main().

```
int score;
int bonus = 0;
cin >> score;
if (score >= 50)
{
  bonus += 5;
  cout << "Pass\n";
 }
if (score >= 80)
 {
  bonus += 3;
  cout << "Excellent\n";
 }
cout << "Bonus = " << bonus << "\n";
```

a) Write the exact output for each input: 49, 50, 79, and 80.

49 =
```
Bonus = 0
```

50 =
```
Pass
Bonus = 5
```

79 =
```
Pass
Bonus = 5
```

80 =
```
Excellent
Bonus = 8
```

b) For an input of 80, How many if bodies execute? Explain why.
2 bodies will be executed.

c) Does the final cout statement execute when input is 49? Explain.
Yes. The program will skip all bodies and output with no bonus point.

Exercise 3 : Find and fix the logic errors.

a) Low-Stock Warning. Display low stock only when stock is less than 5. Always display check complete. Test with 4 and 5.

```
int stock;
cin >> stock;
if (stock < 5);
      cout << "Low stock\n";
cout << "Check complete\n";
```

Answer:

```
int stock;
cin >> stock;
if (stock < 5)
  {
      cout << "Low stock\n";
  }
cout << "Check complete\n";
```

Test:

4 =	Low stock
5 =	Check complete

b) PIN check. Only when the PIN equals 2468, display Access granted and set attempts to 0. For any other PIN, attempts must remain 3. Test both cases.

```
int pin, attempts = 3;
cin >> pin;
if (pin = 2468)
      cout << "Access granted\n";
      attempts = 0;
cout << "Attempts left = " << attempts << "\n";
```

Answer:

```
int pin, attempts = 3;
cin >> pin;
if (pin == 2468)
      cout << "Access granted\n";
      attempts = 0;
cout << "Attempts left = " << attempts << "\n";
```

Test:

Pin - 2468 =	Access granted
Any pin =	Attempts left = 3

c) Valid Mark. Display Valid mark only for marks from 0 to 100, inclusive. Test with -1, 0, 100 and 101.

```
int mark;
cin >> mark;
if (0 <= mark <= 100)
      cout << "Valid mark\n";
```

Answer:

```
int mark;
cin >> mark;
```

```
if (0 <= mark && mark <= 100)
        cout << "Valid mark\n";
```

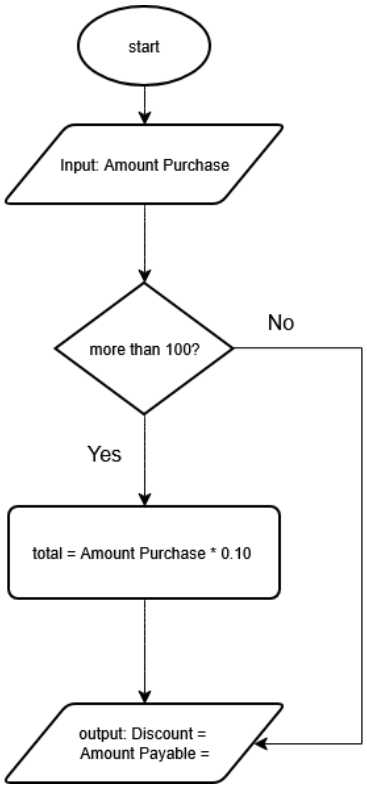
Test:

-1 =	`No Output`
0 =	`Valid mark`
100 =	`Valid mark`
100 =	`No Output`

Exercise 4: Campus shop Discount

A campus shop gives a 10% discord when a purchase is Rm100 or more. For Smaller purchase, the discount is RM0.00. Assume the purchase amount is valid non-negative number.

a) Prepare pseudocode or a flowchart for the solution.



b) Write a C++ program that reads the purchase amount, initialises the discount to zero, and uses one if statement to calculate any discount.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
	double total = 0.00;
	double amount = 0.00;
	double discount = 0.00;

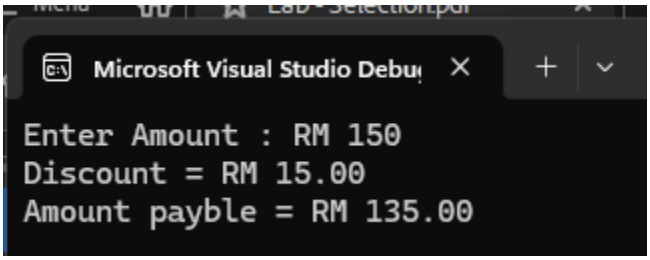
	cout << "Enter Amount : RM ";
	cin >> amount;

	if (amount > 100)
	{
		discount = amount * 0.10;
		total = amount - discount;
	}
	amount = total;

	cout << fixed << setprecision(2);
	cout << "Discount = RM " << discount << endl;
	cout << "Amount payble = RM " << total << endl;

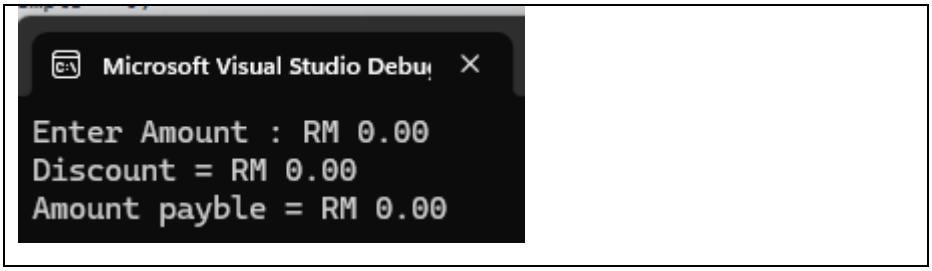
	return 0;
}
```

c) Calculate and display the discount and final amount payable to two decimal places. Use <iomanip> for money formatting.



d) Test your program with RM0.00, RM99.99, RM100.00, and RM150.00. Record the expected and actual results.

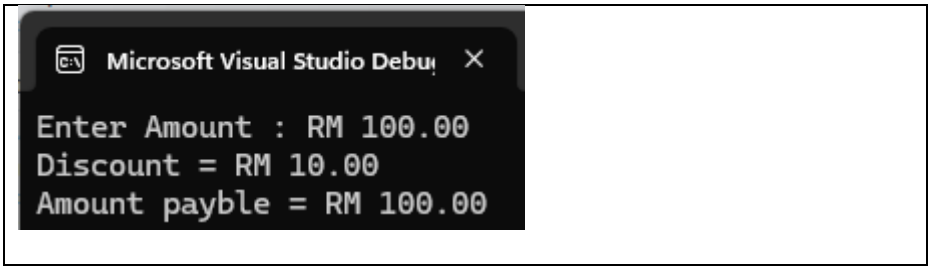
RM0.00 =



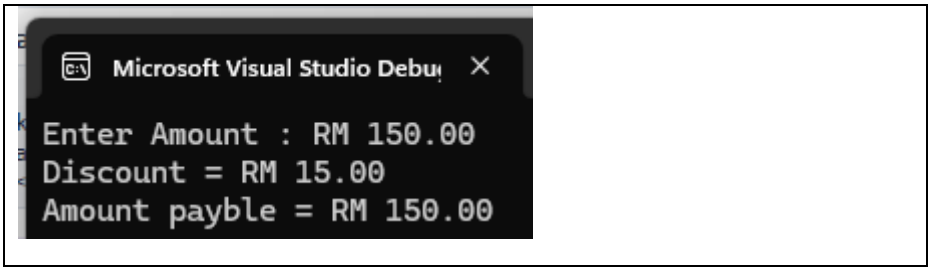
RM 99.99 =



RM 100.00 =



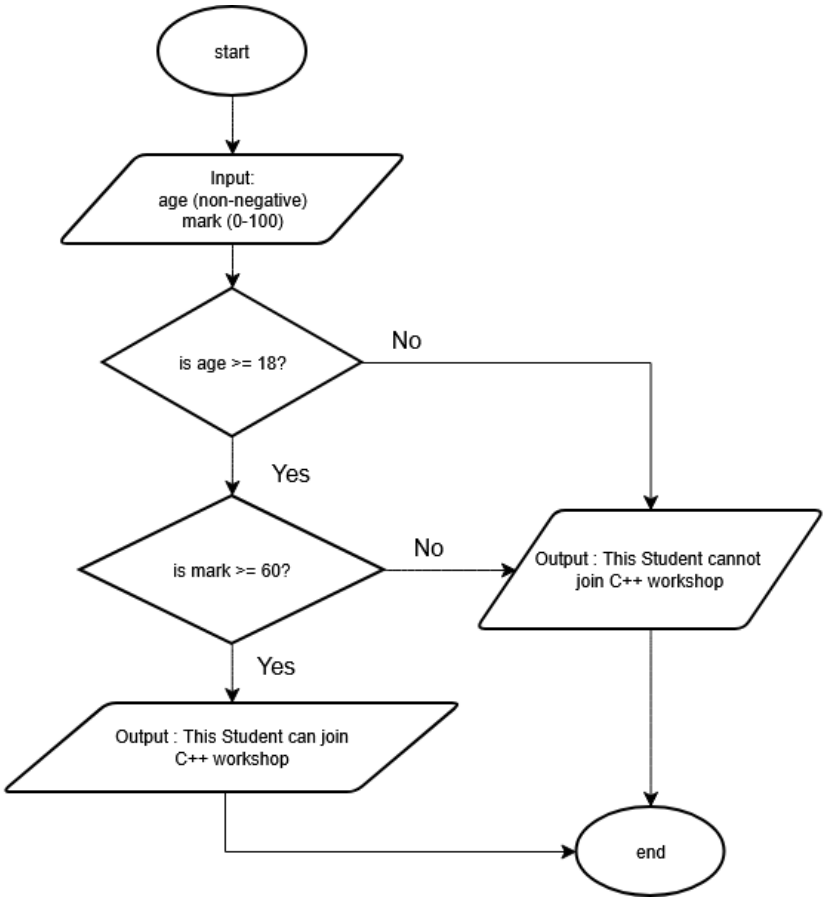
RM 150.00 =



Exercise 5: Workshop eligibility checker

A student can join a C++ workshop both conditions are met: the student at least 18 years old and has a prerequisite test mark of at least 60. Assume the age is non-negative and the mark is an integer from 0 to 100.

a) Prepare pseudocode or a flowchart for the solution.



b) Read the student's age and test mark. Declare a bool variable named eligible and assign it the result of a logical expression.

```
Int age;
Int mark;
Bool eligible;

Cin >> age;
Cin >> mark;

Eligible = (age >= 18 && mark >= 60)
```

c) Use if (eligible) to display **both** lines below in one body. Use braces.

```
If (eligible)
{
   Cout <<  “Eligible for the workshop. \n”;
   Cout << “Please complete the registration form.\n”;
}
```

d) Use a separate if (!eligible) statement to display Not eligible for the workshop.

```
If (!eligible)
{
   Cout <<  “Not Eligible for the workshop.”;
}
```

e) Test all four combinations below. Record the output and state which condition prevents eligibility in each unsuccessful case.

Age	Test Mark	Expected Output / Reason
17	59	Not Eligible for the workshop. Reason : Test Mark and age Not Meet Condition
17	60	Not Eligible for the workshop. Reason : age Not Meet Condition
18	59	Not Eligible for the workshop. Reason : Test Mark Not Meet Condition
18	60	Eligible for the workshop. Reason : Both age and Test Mark are Meet Condition

Exercise 6 : Optional : Monthly Payment Validator

A shop lets customer divide a purchase into equal monthly payments, without interest. Write a C++ program that reads the total purchase amount and the number of months as a double and an integer, respectively.

a) Declare a bool variable named validPayment. Assign it a logical expression that is true only when the total is **zero or greater** and months is **greater than zero**.

```
Bool validPayment;

validPayment = (total >= 0 && months > 0)
```

b) When the input is valid, calculate monthlyPayment = total / months and display the payment to two decimal places.

```
if (validPayment)
{
        double monthlyPayment = total / months;
        cout << fixed << setprecision(2);
        cout << "Monthly payment = RM " << monthlyPayment << endl;
}
```

c) When the input is invalid, display Invalid payment details. Use two separate if statements. The division must occur only for valid input.

```
If(!validPayment)
{
      Cout << “Invalid Payments Details”
}

if (validPayment)
{
        double monthlyPayment = total / months;
        cout << fixed << setprecision(2);
        cout << "Monthly payment = RM " << monthlyPayment << endl;
}
```

d) Explain why zero months must be rejected before the calculation.

Total (RM)	Months	Expected result
1200.00	6	Monthly payment = RM 200.00
1200.00	0	Invalid payment details.
1200.00	-3	Invalid payment details.
-100.00	6	Invalid payment details.
0.00	6	Monthly payment = RM 0.00

Zero months must be rejected because division by zero is undefined and will cause an error in the calculation. Therefore, months must be greater than zero before calculating the monthly payment.